

GUION DE PRÁCTICA — VIDEO TÉCNICO

Plan de Mejoramiento — Competencia Seguridad Informática

Manuela Córdoba Robledo · Identificación 1015071897 · Ficha 3229209

Sistema: SecureDesk ADSO — API de Gestión de Incidentes de Seguridad

CÓMO USAR ESTE DOCUMENTO

Esto es para PRACTICAR en voz alta, no para leer palabra por palabra mirando la pantalla mientras grabas — el plan de mejoramiento dice explícitamente que no basta con leer diapositivas. Léelo varias veces, repítelo con tus palabras, y graba cuando te salga natural. Lo importante es que digas las ideas, no el texto exacto.

Duración total estimada: 17-18 minutos (dentro del rango 15-20 que pide el plan).

0. Introducción (0:00 - 0:40)

QUÉ MOSTRAR EN PANTALLA

- Tu cara frente a la cámara, o el README del repositorio en GitHub.

QUÉ DECIR (practica hasta que te salga natural, no lo leas fijo)

Hola, mi nombre es Manuela Córdoba Robledo, soy aprendiz de Tecnología en Análisis y Desarrollo de Software del SENA, ficha 3229209. Este video es la sustentación técnica de mi plan de mejoramiento para la competencia de Seguridad Informática.

En los próximos minutos les voy a mostrar el problema que resolví, la arquitectura del sistema, cómo funciona la autenticación y la autorización por roles, los controles de seguridad que implementé, la estructura del código, las pruebas que ejecuté con sus resultados, los riesgos que todavía quedan pendientes, y voy a cerrar con una demostración en vivo del sistema funcionando.

1. El problema (0:40 - 2:30)

QUÉ MOSTRAR EN PANTALLA

- El documento técnico PDF, sección 1 (Descripción del sistema).

QUÉ DECIR (practica hasta que te salga natural, no lo leas fijo)

El sistema que trabajé se llama SecureDesk ADSO, una API para gestionar incidentes de seguridad. Se eligió este tipo de sistema porque obliga a resolver, en un solo backend, los mismos problemas de cualquier implantación real de software: quién puede entrar, qué puede ver cada persona según su rol, cómo se valida la información que llega, cómo se deja rastro de lo que pasa, y cómo se recupera el sistema si algo sale mal.

Quiero ser honesta desde el principio con la metodología que seguí: las clases 1, 2 y 6 de este curso las documenté trabajando sobre RitmoApp, mi proyecto de grado, porque en ese momento estábamos definiendo arquitectura y roles junto con mi equipo. Pero la implementación técnica real —un backend corriendo de verdad, con base de datos, pruebas ejecutadas y evidencia reproducible— la construí sobre SecureDesk ADSO, la práctica de referencia de la competencia. Todo lo que les voy a mostrar en este video, con código y pruebas en vivo, es sobre SecureDesk ADSO.

2. Arquitectura (2:30 - 4:30)

QUÉ MOSTRAR EN PANTALLA

- VS Code: árbol de archivos (backend/, frontend/).
- Abrir backend/src/app.js.
- Abrir backend/prisma/schema.prisma.

QUÉ DECIR (practica hasta que te salga natural, no lo leas fijo)

SecureDesk ADSO está organizado en tres capas separadas: el backend en Node.js con Express, la base de datos PostgreSQL administrada con Prisma como ORM, y un frontend en HTML, CSS y JavaScript, separado del backend, que consume la API por HTTP.

Aquí en app.js pueden ver el orden de los middlewares: primero CORS, después el parser de JSON con un límite de un megabyte, luego Morgan para registrar cada petición, después se montan las rutas, y al final el manejo de errores.

Y aquí está el modelo de datos: un modelo User con un enum de Role que define los tres roles —ADMIN, ANALISTA y CONSULTA—, y un modelo Incident que representa cada incidente de seguridad, con su severidad, su estado y quién lo reportó.

3. Autenticación (JWT) (4:30 - 6:30)

QUÉ MOSTRAR EN PANTALLA

- Abrir backend/src/routes/auth.routes.js y backend/src/middlewares/auth.js.
- DEMO EN VIVO: login como admin desde el frontend (localhost:5500); mostrar el token en las herramientas de desarrollador del navegador (pestaña Network).

QUÉ DECIR (practica hasta que te salga natural, no lo leas fijo)

La autenticación la resolví con JSON Web Tokens, JWT. Cuando alguien inicia sesión en /auth/login, el sistema busca el usuario por su correo, compara la contraseña enviada contra el hash guardado con bcrypt —nunca se guarda la contraseña en texto plano—, y si coincide, firma un token que incluye el id del usuario, su correo y su rol, con una duración de dos horas.

Este middleware, authenticateJWT, se ejecuta en cada ruta protegida: toma el token del encabezado Authorization, lo verifica, y si es válido deja pasar la petición; si no, la rechaza antes de que llegue a cualquier otro código.

Se los muestro en vivo: voy a iniciar sesión como administrador desde el frontend.

4. Autorización por rol (RBAC) (6:30 - 9:00)

QUÉ MOSTRAR EN PANTALLA

- Abrir backend/src/middlewares/authorizeRoles.js.
- DEMO EN VIVO con curl o Postman contra /admin/ping: 1) sin token, 2) con token CONSULTA, 3) con token ADMIN.

QUÉ DECIR (practica hasta que te salga natural, no lo leas fijo)

La autorización por roles la implementé con un middleware separado, authorizeRoles. Recibe la lista de roles permitidos para una ruta y la compara contra el rol que viene dentro del token. Si el rol no está permitido, responde 403; si el token ni siquiera existe o es inválido, eso ya lo detiene authenticateJWT antes, con un 401.

Esto es importante y lo quiero dejar muy claro: la autorización se valida en el backend, no solo ocultando botones en la interfaz. Si alguien con rol CONSULTA intentara llamar directamente a la API con Postman o con curl, sin pasar por mi frontend, seguiría recibiendo 403.

Se los demuestro ahora mismo con la ruta /admin/ping: primero sin ningún token, después con el token de un usuario CONSULTA, y por último con el token de ADMIN.

5. Controles de seguridad (9:00 - 11:30)

QUÉ MOSTRAR EN PANTALLA

- Abrir backend/src/validators/incident.schema.js.
- Abrir el rate limiter en auth.routes.js.
- Abrir backend/.env (sin mostrar valores reales de cerca) y .gitignore.

QUÉ DECIR (practica hasta que te salga natural, no lo leas fijo)

Además de autenticación y autorización, implementé varios controles más, y para cada uno digo si es preventivo, detective o correctivo.

La validación de datos con Zod es un control preventivo: revisa que el título, la descripción, la severidad y el correo cumplan un formato válido antes de guardarse. Si algo está mal, responde 400 con el detalle exacto de cada campo.

El límite de intentos de login con express-rate-limit es otro control preventivo: permite solo cinco intentos por minuto en /auth/login, para frenar un ataque de fuerza bruta.

Morgan, que registra cada petición HTTP, es un control detective: no evita nada, pero deja un rastro para investigar después qué pasó.

Y el manejo centralizado de errores es un control correctivo: cualquier error se convierte en una respuesta JSON consistente, sin exponer detalles internos del servidor.

Por último, la gestión de secretos: el archivo .env, con la contraseña real de la base de datos y el secreto JWT, nunca se sube al repositorio —está excluido en el .gitignore—. Lo que sí se publica es un .env.example, una plantilla sin valores reales.

6. Estructura del código y repositorio (11:30 - 13:00)

QUÉ MOSTRAR EN PANTALLA

- GitHub: repositorio securedesk-adso, estructura de carpetas, README, ramas master y plan-mejoramiento, historial de commits.

QUÉ DECIR (practica hasta que te salga natural, no lo leas fijo)

Todo el código, la documentación y las evidencias están publicados en GitHub. Aquí ven la estructura de carpetas: backend, frontend, evidencia con las pruebas reales, y el documento técnico completo. El repositorio tiene dos ramas: master, y plan-mejoramiento, que es la rama específica para esta entrega, tal como lo pide la guía. También está el historial de commits y el README con las instrucciones de instalación.

7. Pruebas ejecutadas y resultados (13:00 - 15:30)

QUÉ MOSTRAR EN PANTALLA

- Carpeta evidencia/ (mostrar algunos .txt).
- DEMO EN VIVO del ciclo completo: bash scripts/backup.sh → borrar datos a propósito → bash scripts/restore.sh → verificar que volvieron.

QUÉ DECIR (practica hasta que te salga natural, no lo leas fijo)

Ahora les muestro las pruebas que ejecuté, todas contra el sistema corriendo de verdad, no sobre un diseño teórico. En esta carpeta están las salidas reales: el login por cada rol, las respuestas 401, 403 y 200 que ya mostré, la validación de Zod rechazando datos inválidos, y el rate limiting activándose después del cuarto intento fallido.

La prueba más importante para mí es la de backup y restauración, porque demuestra que el sistema se puede recuperar de verdad. Se las muestro en vivo: primero genero un backup real de la base de datos, después borro a

propósito toda la tabla de incidentes para simular un daño, y por último restauro desde el backup que acabo de generar, y verificamos que los tres incidentes originales volvieron exactamente igual.

8. Riesgos pendientes / hallazgos (15:30 - 17:00)

QUÉ MOSTRAR EN PANTALLA

- Documento PDF, sección 21 (Hallazgos).

QUÉ DECIR (practica hasta que te salga natural, no lo leas fijo)

Quiero ser honesta con lo que todavía no está resuelto, porque documentar solo lo que funciona no sería un análisis de seguridad real. Tengo tres hallazgos abiertos: primero, el JWT_SECRET que estoy usando es un valor de desarrollo, no uno generado para producción —si alguien lo llegara a conocer, podría falsificar tokens de cualquier rol—. Segundo, no tengo configurado HTTPS, así que todo el tráfico corre en texto plano sobre HTTP local. Y tercero, no ejecuté una prueba de carga real con muchos usuarios al mismo tiempo, solo peticiones individuales.

Ya tengo un plan de corrección con fechas para cerrar los dos hallazgos de impacto alto antes de exponer este sistema a datos reales.

9. Cierre (17:00 - 18:30)

QUÉ MOSTRAR EN PANTALLA

- Tu cara, o el README del repositorio.

QUÉ DECIR (practica hasta que te salga natural, no lo leas fijo)

Para cerrar: construir SecureDesk ADSO práctica por práctica, en vez de documentar controles hipotéticos sobre un sistema que no existe, me dejó claro que la seguridad de una API no depende de un solo control grande, sino de varios controles pequeños que se apoyan entre sí. La autenticación no sirve de nada sin autorización por rol, la autorización no sirve de nada si los datos no se validan, y ningún control anterior importa si un error humano borra la base de datos y no hay cómo restaurarla.

El sistema todavía no está listo para producción —y ya expliqué por qué—, pero entiendo perfectamente qué falta y cómo resolverlo. Todo el código, la documentación y las evidencias están en el repositorio que comparto en la descripción. Muchas gracias, y quedo disponible para la sustentación.

Checklist antes de subir el video

- ☐ Duración entre 15 y 20 minutos.
- ☐ Se ve tu cara o se escucha tu voz explicando con naturalidad (no lectura robótica).
- ☐ Se muestra código real en pantalla, no solo diapositivas.
- ☐ Incluye al menos las dos demos en vivo: RBAC (401/403/200) y backup/restauración.
- ☐ Mencionas explícitamente qué riesgo o control demuestra cada parte.
- ☐ Subir a YouTube como oculto o público.
- ☐ Pegar el link en el documento PDF (reemplazar el placeholder) y en el README.